Software Engineering Lab

Lab 17

Architecture Decision Record

Kunci Silsilah Desain,
Validasi Immutability Git



lukman-ss

Masalah: Konteks Arsitektur yang Hilang di Masa Depan

Tanpa catatan tertulis, alasan di balik keputusan teknis penting lenyap saat tim berganti.

MOTIVASI

Dokumen desain di wiki internal cepat usang (documentation drift) dan terpisah dari kode sumber aktual. Ketika insinyur baru bergabung, latar belakang arsitektur tidak lagi dapat dilacak.

DAMPAK

Tim mengulangi perdebatan teknis yang sama berulang kali atau membatalkan pola masa lalu secara terburu-buru tanpa memahami batasan awal yang memicunya.

Architecture decisions must form an immutable, verifiable directed acyclic graph.

Mental Model: Keputusan Sebagai Kode Sumber di Git

Menyimpan ADR bersama kode memastikan review terintegrasi dan silsilah tidak terputus.

KO-LOKASI DENGAN KODE

ADR disimpan dalam format Markdown langsung di repositori Git (docs/adr/). ADR berevolusi melalui mekanisme Pull Request dan code review yang sama dengan kode aplikasi.

IMMUTABILITY KEPUTUSAN

Sekali disetujui, isi ADR tidak boleh disunting retroaktif untuk mengubah arah keputusan. Perubahan arsitektur dituangkan ke dalam ADR baru yang menunjuk ADR lama.

Record decisions in the repository where code lives, not in external disconnected wikis.

Batasan: Kapan Keputusan Memerlukan ADR?

ADR hanya dicadangkan untuk keputusan yang signifikan secara arsitektural.

TERMASUK KEPUTUSAN ARSITEKTUR

- Struktur: Pemisahan domain dan batas modularitas
- NFR: Skalabilitas, ketersediaan, toleransi beban
- Dependensi: Pemilihan basis data, pustaka inti, atau runtime

DI LUAR CAKUPAN ADR

- Refaktorisasi metode internal rutin
- Standar penamaan variabel atau formatting
- Perbaikan bug lokal tanpa dampak struktural

Document architectural significance, not routine tactical code modifications.

Lima Seksi Standar Dokumen MADR

Struktur terstandarisasi memastikan setiap catatan menangkap konteks secara utuh.

TIGA SEKSI AWAL

- Title: Header H1 bernomor urut dan judul keputusan
- Status: Status siklus hidup formal (Proposed/Accepted/dll)
- Context: Latar belakang masalah, batasan, dan forces

DUA SEKSI AKHIR

- Decision: Kalimat aktif perubahan desain yang disepakati
- Consequences: Rincian dampak positif, trade-off negatif, dan risiko netral yang harus ditanggung tim

Standardized document anatomy guarantees completeness and prevents missing context.

Model Data Record dan Status Diskret di Go

Representasi memori dokumen ADR setelah dipindai oleh parser.

STRUKTUR DATA MEMORI

```
type Record struct {
    ID           int
    Title        string
    Status       Status
    SupersededBy int
    Supersedes   int
    Content      string
}
```

TIPE STATUS DISKRET

```
type Status string

const (
    StatusProposed   Status = "Proposed"
    StatusAccepted   Status = "Accepted"
    StatusSuperseded Status = "Superseded"
    StatusDeprecated Status = "Deprecated"
    StatusRejected   Status = "Rejected"
)
```

Typed structs and discrete status constants enforce compile-time domain integrity.

Siklus Hidup: State Machine Lima Status

Mengatur alur mutasi status dari usulan hingga penghentian keputusan.

FASE AKTIF

- Proposed: Usulan arsitektur dalam tahap peninjauan tim
- Accepted: Keputusan resmi aktif yang mengikat desain sistem

FASE TERMINAL

- Superseded: Keputusan lama digantikan oleh ADR baru
- Deprecated: Keputusan ditinggalkan tanpa pengganti
- Rejected: Usulan ditolak secara formal untuk cegah debat ulang

Clear state transitions prevent arbitrary informal statuses from creeping into Git.

Parser Markdown: Scanning Linier Hemat Memori

Membaca dokumen baris demi baris menggunakan bufio.Scanner tanpa parsing AST berat.

ALUR SCANNER

```
scanner := bufio.NewScanner(reader)
for scanner.Scan() {
    line := strings.TrimSpace(scanner.Text())
    // Ekstraksi regex per baris
}
```

EFISIENSI KOMPUTASI

Scanning baris per baris hemat alokasi heap dan beroperasi deterministik, memproses ratusan berkas ADR dalam hitungan milidetik tanpa dependensi library Markdown eksternal.

Linear stream parsing extracts metadata with zero AST overhead.

Ekstraksi Regex: Judul dan Nomor Monotonik

Memastikan judul tingkat satu mematuhi konvensi penomoran baku.

POLA REGEX JUDUL

```
var titleRegex = regexp.MustCompile(
    `^#\s+(\d+)\.\s+(.+)$`)
matches := titleRegex.FindStringSubmatch(line)
if matches ≠ nil {
    record.ID, _ = strconv.Atoi(matches[1])
    record.Title = matches[2]
}
```

VALIDASI STRUKTURAL

Setiap ADR wajib memiliki header H1 dengan nomor urut positif dan titik pemisah. Judul tanpa nomor langsung digagalkan dengan error deskriptif.

Regular expressions enforce strict header formatting before domain processing.

Parsing Silsilah: Relasi Supersedes & Superseded

Mengekstrak tautan penggantian timbal-balik antar dokumen ADR.

SUPERSEDED BY METADATA

```
Status: Superseded by 2
→ record.Status = StatusSuperseded
→ record.SupersededBy = 2
```

SUPERSEDES METADATA

```
Supersedes: 1
→ record.Supersedes = 1
```

Lineage references build the nodes and edges of the decision dependency graph.

Model Linier vs Directed Acyclic Graph (DAG)

Bagaimana evolusi keputusan membentuk pohon silsilah berarah.

RANTAI EVOLUSI KEPUTUSAN

Keputusan arsitektur tidak berdiri sendiri secara linier. ADR 1 digantikan ADR 3, yang kelak digantikan ADR 7. Setiap simpul mencatat konteks pada masanya.

VISIBILITAS ARSITEKTUR

Insinyur baru dapat menelusuri riwayat evolusi dari keputusan fondasi hingga konfigurasi operasional aktif saat ini tanpa kebingungan.

Architecture decisions evolve as a directed acyclic graph, not a disconnected flat list.

Empat Skenario Kerusakan Dokumen Manual

Jebakan yang tak terhindarkan jika repositori Markdown dikelola tanpa linter otomatis.

ANOMALI REFERENSI

- Dangling Reference: Menyatakan "Superseded by 99", tetapi ADR 99 tidak ada
- One-Sided Supersession: ADR 2 mengklaim "Supersedes 1", tetapi ADR 1 masih berstatus Accepted

ANOMALI URUTAN & STATUS

- Non-Monotonic: Penomoran melompat (1, 2, 4 tanpa ADR 3)
- Invalid Status: Penggunaan status informal (Draft/Pending)

Without automated linting, documentation repositories rapidly degrade into broken links.

Linter: Penegakan Penomoran Monotonik

Memastikan rangkaian identitas ADR utuh tanpa celah penomoran.

ALGORITME VALIDASI URUTAN

```
sort.Slice(records, func(i, j int) bool {
    return records[i].ID < records[j].ID
})
for i, r := range records {
    if r.ID ≠ i+1 {
        errs = append(errs, errNonMonotonic)
    }
}
```

JAMINAN INTEGRITAS GIT

Memverifikasi tidak ada berkas ADR yang hilang, tertinggal, atau memiliki nomor ganda yang memicu kerancuan historis.

Monotonic sequence enforcement guarantees no architectural record is lost in Git history.

Validasi Timbal-Balik (Bidirectional Lineage)

Mengunci relasi dua arah antara keputusan lama dan keputusan penggantinya.

SISI ADR PENGGANTI

Jika ADR B mendeklarasikan "Supersedes: A", linter memverifikasi bahwa ADR A benar-benar berstatus Superseded dan menunjuk ke B.

SISI ADR TERGANTIKAN

Jika ADR A berstatus "Superseded by B", linter memverifikasi bahwa ADR B benar-benar ada dan mendeklarasikan "Supersedes: A".

Bidirectional reciprocal validation guarantees that decision graph edges are strictly symmetrical.

Konkurensi: Goroutine Memvalidasi Graf Simpul

Memanfaatkan konkurensi native Go untuk validasi graf super cepat.

PEMBAGIAN GOROUTINE

```
var wg sync.WaitGroup
for _, r := range records {
    wg.Add(1)
    go func(rec *Record) {
        defer wg.Done()
        // Validasi graf simpul timbal-balik
    }(r)
}
wg.Wait()
```

LATENSI MINIMAL

Setiap simpul divalidasi secara paralel terhadap lookup map di memori. Ratusan berkas divalidasi dalam hitungan mikrodetik.

Parallelize node validation across Go routines for instant verification of large ADR trees.

Keamanan Thread: Mutex Mengunci Agregasi Error

Mencegah data race saat goroutine menemukan pelanggaran secara konkuren.

MUTEX SINKRONISASI

```
var mu sync.Mutex

if err := validateLineage(rec); err ≠ nil {
    mu.Lock()
    errs = append(errs, err)
    mu.Unlock()
}
```

THROUGHPUT BERSIH

Koleksi error bersama aman dari manipulasi pointer konkurensi. Terbukti bersih dari data race saat diuji dengan go test -race.

Mutex synchronization around shared error collectors ensures thread-safe diagnostic aggregation.

Cakupan Test Suites: 6 Assertion Parser

Pengujian mendalam pada tests/parser_test.go membuktikan ketangguhan parsing.

TEST POSITIF

- Proposed Doc: ID=1, Status=StatusProposed
- Accepted Doc: ID=2, Status=StatusAccepted
- Superseded Lineage: Ekstraksi SupersededBy=2 & Supersedes=1

TEST NEGATIF

- Invalid Title: Menolak header tanpa nomor urut bulat
- Unknown Status: Menolak status di luar 5 status baku
- Empty Content: Menangani dokumen kosong dengan aman

Exhaustive parser tests verify both standard document parsing and graceful error rejection.

Cakupan Test Suites: 8 Assertion Linter

Pengujian pada tests/linter_test.go membuktikan seluruh skenario anomali graf.

TEST INTEGRITAS PENOMORAN

- Monotonic Valid: Urutan 1..N sempurna (PASS)
- Missing ID: Mendeteksi nomor lompat (1, 3)
- Duplicate ID: Menolak dua file berebut nomor sama

TEST SILSILAH TIMBAL BALIK

- Dangling Target: Menangkap Superseded by 99
- One-Sided Supersede: Menggagalkan klaim sepihak
- Concurrent Clean: Nol data race pada 100 record paralel

Multi-faceted linter assertions catch structural drift before changes reach production.

tests/linter_test.go:20-110

Pemisahan Tanggung Jawab: Parser vs Linter

Menerapkan Single Responsibility Principle pada perancangan validator dokumen.

TANGGUNG JAWAB PARSER

Fokus murni memetakan teks Markdown mentah menjadi struct Record tanpa menghakimi validitas hubungan antardokumen.

TANGGUNG JAWAB LINTER

Fokus murni memvalidasi aturan bisnis agregat (urutan ID monotonik dan simetri graf referensi) dari slice []Record.

Decouple syntax token parsing from whole-graph semantic rule evaluation.

Nilai Strategis Status Rejected

Mengapa mencatat usulan yang ditolak sama pentingnya dengan keputusan yang diterima.

PENGHEMATAN WAKTU TIM

Menghentikan perdebatan teknis berulang ("mengapa kita tidak memakai NoSQL?") setiap kali anggota tim baru bergabung.

DOKUMENTASI BUKTI EVALUASI

Menyimpan batasan beban atau hasil benchmark yang membuat opsi tersebut gugur, menjaga konsistensi arah arsitektur.

Documenting rejected architectural proposals saves hundreds of engineering hours in cyclical debates.

CLI Demo: Integrasi Langsung ke File System

cmd/demo/main.go memindai berkas Markdown nyata di folder docs/adr/.

PEMINDAIAN DIREKTORI

```
files, _ := filepath.Glob("docs/adr/*.md")
for _, f := range files {
    content, _ := os.ReadFile(f)
    rec, _ := adr.Parse(string(content))
    records = append(records, rec)
}
linter := adr.NewLinter()
errs := linter.Lint(records)
```

PINTU GERBANG CI/CD

Mengembalikan exit code 1 jika ditemukan inkonsistensi, memblokir pull request yang merusak silsilah keputusan Git.

Simple CLI binaries easily integrate into Git pre-commit hooks and CI/CD pipelines.

Batas Bukti Implementasi Lab 17

Kejujuran teknis mengenai ruang lingkup pengujian yang dibuktikan dalam lab.

APA YANG DIBUKTIKAN

- Algoritma parsing Markdown MADR lokal
- Validasi urutan ID monotonik (1..N)
- Validasi timbal-balik silsilah graf konkuren

DI LUAR BATAS LAB

- Tidak mencakup webhook GitHub Actions otomatis
- Tidak mem-parsing format AsciiDoc
- Tidak membangun visualizer diagram interaktif

A rigorous engineering laboratory clearly demarcates its proof boundaries from production wrappers.

Checklist Kualitas ADR Tim Engineering

Panduan praktis bagi tim sebelum melakukan merge dokumen keputusan baru.

CHECKLIST FORMAT

1. Apakah 5 seksi wajib (Title, Status, Context, Decision, Consequences) lengkap?
2. Apakah ID bertambah monotonik tanpa melompati nomor?
3. Apakah status menggunakan 5 status formal?

CHECKLIST SILSILAH

4. Jika menggantikan keputusan lama, apakah tautan Supersedes dan Superseded by terpasang timbal-balik?
5. Apakah ADR yang ditolak mencatat argumen penolakan?

Systematic architectural checklists ensure decision records remain durable and auditable over decades.

Kunci Silsilah Desain, Validasi Immutability Git.

Architecture decisions must form an immutable, verifiable directed acyclic graph.

> Software Engineering Lab 17 — Architecture Decision Record
>
> internal/adr/models.go • internal/adr/parser.go • internal/adr/linter.go
> tests/parser_test.go • tests/linter_test.go • cmd/demo/main.go • README.md
>
> REPOSITORY
> github.com/lukman-ss/software-engineering-lab/tree/main/labs/17-architecture-decision-record
>
> **Lukman / lukman-ss**
>
> Sumber: commit eb612371e53183397649b7091a33d4068f70a7cb • Verified Go 1.22 (Race-Clean)